ISLS Extension Phase 8 v1.0.0

C27: The FOUNDRY

Closed-Loop Software Fabrication

Compile → Test → Fix → Until It Works

+ Workspace Intelligence

The crate that makes the Forge
actually produce working software.

Sebastian Klemm

18 March 2026

Abstract

This document specifies `isls-foundry` (C27), the closed-loop fabrication engine that bridges the gap between "the Forge generates code strings" and "the operator has a working, compilable, tested software project on disk."

The Foundry wraps the entire Forge pipeline (C21–C26) in a build-test-fix loop: generated code is written to a real project directory, compiled by the real toolchain (`cargo`), tested by the real test runner, and — if compilation or tests fail — the error output is fed back to the Oracle (C25) as a correction prompt. The loop iterates until the project compiles cleanly, tests pass, or the retry budget is exhausted.

Additionally, the Foundry provides Workspace Intelligence: the ability to understand an existing codebase, identify what exists, what is missing, and generate code that fits into the existing project rather than creating a new one from scratch. This enables incremental development: "add an endpoint," "implement this trait," "fix this module" — not just "generate a whole new project."

The Foundry is the layer that turns ISLS from a formal system that happens to generate code into a practical software development tool.

Status: Implementation Specification.

Parent: ISLS v1.0.0 + Extensions Phase 1–7.
Depends on: C23 (Forge), C24 (Compose), C25 (Oracle), C26 (Templates).

Contents

I Why This Changes Everything 5

1 The Gap 5

2 Axioms 5

II The Build-Test-Fix Loop 5

3 Loop Architecture 6

4 Correction Prompts 6

5 Toolchain Integration 7

III Project Scaffolding 8

6 Complete Project Generation 8

IV Workspace Intelligence 9

7 Understanding Existing Code 9

8 Incremental Operations 11

V Quality Gates 12

9 The Extended Validation Pipeline 12

VI Fabrication Modes 12

10 Interactive Mode 13

11 Batch Mode 13

12 Headless Mode (for automation) 14

VII Rust API 14

VIII Gateway Integration 15

IX Acceptance Tests 16

X Handoff **17**

13 Test Count **17**

14 Completion Criteria **17**

Part I
Why This Changes Everything

1 The Gap

Without the Foundry, the Forge pipeline is:

```
DecisionSpec -> PMHD -> ArtifactIR -> Oracle -> String -> Crystal
```

The output is a string that *claims* to be Rust code. But nobody checked. The string might have syntax errors, missing imports, wrong types, incompatible trait bounds, or simply not compile. The 8-gate cascade validates the crystal's *formal* properties (hash integrity, evidence chain, etc.), not the *semantic* correctness of the code inside it.

With the Foundry, the pipeline becomes:

```
DecisionSpec -> PMHD -> ArtifactIR -> Oracle -> String
  -> Write to disk
  -> cargo check        (does it compile?)
  -> cargo test         (do tests pass?)
  -> cargo clippy       (is it idiomatic?)
  -> IF FAIL: feed error to Oracle, iterate
  -> IF PASS: Crystal (now proven to compile and test)
```

The crystal now means: "this code compiles, its tests pass, and it has been formally validated." That is a qualitatively different guarantee.

2 Axioms

Axiom 2.1 (Compilation is Validation)**.** A code artifact that does not compile is not valid, regardless of its formal crystal properties. The Foundry adds `cargo check` as a *ninth gate* for Rust artifacts. An artifact that passes the 8 formal gates but fails compilation is rejected.

Axiom 2.2 (Tests are Proof)**.** Generated code **MUST** include tests. A code artifact without tests is structurally incomplete. The Foundry requires at least one test per public function/method and verifies that all generated tests pass.

Axiom 2.3 (Errors are Feedback)**.** Compiler errors and test failures are not terminal — they are information. The Foundry feeds error output back to the Oracle as structured correction prompts. Each retry cycle narrows the error space.

Axiom 2.4 (Workspace is Context)**.** The Forge does not operate in a vacuum. It operates in the context of a real project directory with existing code, existing types, existing tests. The Foundry provides this context to the Oracle so that generated code fits into what already exists.

Part II
The Build-Test-Fix Loop

3 Loop Architecture

Algorithm 1 FOUNDRY_FABRICATE: Closed-loop fabrication

Require: ForgeResult F (from C23), ProjectDir D, Config θ
Ensure: FabricationResult (success with working project, or failure with diagnostics)

```
attempt ← 0
errors ← []
repeat
    attempt ← attempt + 1
    // Step 1: Write files to project directory
    WriteProject(F, D)
    // Step 2: Compile
    compile ← CargoCheck(D)
    if compile.failed then
        errors ← compile.stderr
        F ← OracleCorrect(F, errors, "compile")
        continue
    end if
    // Step 3: Lint
    lint ← CargoClippy(D)
    if lint.warnings > 0 then
        F ← OracleCorrect(F, lint.output, "lint")
        // lint warnings don't block, but attempt correction
    end if
    // Step 4: Test
    test ← CargoTest(D)
    if test.failed then
        errors ← test.stderr
        F ← OracleCorrect(F, errors, "test")
        continue
    end if
    // Step 5: All passed
    return FabricationSuccess(D, F, attempt)
until attempt ≥ θ.max_attempts
return FabricationFailure(D, errors, attempt)
```

4 Correction Prompts

Definition 4.1 (Correction Prompt)**.** When compilation or tests fail, the Foundry constructs a correction prompt that includes:

1. The original code that was generated.
2. The exact compiler/test error output.
3. The specific file and line where the error occurred.
4. The error type classification:
    - `syntax`: parse error, missing semicolons, brackets
    - `type`: type mismatch, wrong generics, missing From impl
    - `import`: missing use statements, wrong module paths

- `trait`: missing trait implementation, wrong bounds
- `lifetime`: borrow checker errors
- `test`: assertion failure, panic in test
- `logic`: test passes but wrong output

5. How many attempts remain (creates urgency for the Oracle to get it right).

```
pub struct CorrectionPrompt {
    pub original_code: String,
    pub error_output: String,
    pub error_file: String,
    pub error_line: Option<usize>,
    pub error_class: ErrorClass,
    pub attempt: usize,
    pub max_attempts: usize,
    pub context: WorkspaceContext, // what else exists in the project
}
```

Definition 4.2 (Error-Aware Oracle Call)**.** The Oracle (C25) receives the correction prompt as an extended synthesis request:

```
System: "You previously generated code that failed to compile.
Fix the error. Return ONLY the corrected complete file.
Do not explain. Do not apologize. Just fix it."

User: "File: src/handlers.rs
Error (attempt 2/5):
error[E0277]: the trait bound `User: Serialize` is not satisfied
  --> src/handlers.rs:42:5
  |
42 |     Json(user)
  |     ^^^^ the trait `Serialize` is not implemented for `User`

Original code:
<full file content>

Context:
<workspace summary showing User struct in models.rs without #[derive(
   Serialize)]>"
```

Remark 4.1. The correction prompt is maximally specific. It does not say "the code has errors." It says exactly what is wrong, exactly where, and provides the full context needed to fix it. This dramatically reduces the Oracle's failure rate on retries. Empirically, most compilation errors are fixed within 1–2 correction cycles.

5 Toolchain Integration

Definition 5.1 (Toolchain Executor)**.** The Foundry shells out to real toolchain commands:

```
pub struct ToolchainExecutor {
    pub cargo_path: String,          // default: "cargo"
    pub rustfmt_path: String,        // default: "rustfmt"
    pub timeout_seconds: u64,        // default: 120
}
```

```
pub struct ToolchainResult {
    pub success: bool,
    pub exit_code: i32,
    pub stdout: String,
    pub stderr: String,
    pub duration_ms: u64,
}

impl ToolchainExecutor {
    pub fn cargo_check(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_build(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_test(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_clippy(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_fmt_check(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_fmt(&self, dir: &Path) -> ToolchainResult;
    pub fn cargo_doc(&self, dir: &Path) -> ToolchainResult;
}
```

Requirement 5.1 (Toolchain Availability)**.** The Foundry **MUST** check toolchain availability at startup. If `cargo` is not found, the Foundry operates in *dry-run mode*: it writes files and skips compilation. The user is informed that compilation verification is disabled.

Part III
Project Scaffolding

6 Complete Project Generation

Definition 6.1 (ProjectScaffold)**.** When the Foundry generates a new project, it produces a complete, runnable Cargo project:

```
project-name/
  Cargo.toml              # workspace or single crate
  .gitignore              # Rust-standard gitignore
  README.md               # generated from DecisionSpec
  src/
    main.rs or lib.rs     # entry point
    <module>.rs            # one file per atom
  tests/
    integration.rs         # integration tests
  examples/                # usage examples (if applicable)
  migrations/              # SQL migrations (if DB template)
  .env.example             # environment variables
```

Definition 6.2 (Cargo.toml Generation)**.** The Foundry generates `Cargo.toml` from the template's dependency list and the Oracle's output:

```
pub struct CargoTomlBuilder;

impl CargoTomlBuilder {
    pub fn build(
        name: &str,
        template: &ArchitectureTemplate,
```

```
        atoms: &[AtomArtifact],
    ) -> String {
        // - Package metadata from DecisionSpec
        // - Dependencies from template + Oracle hints
        // - Features from template configuration
        // - Workspace structure if multi-crate
    }
}
```

Definition 6.3 (README Generation)**.** A README.md is generated from the DecisionSpec and composition tree:

```
# {project-name}

{spec.description}

## Architecture

{composition tree as markdown}

## Getting Started

‘‘‘
cargo build
cargo test
cargo run  # (if binary)
‘‘‘

## Modules

{for each atom: name, purpose, public API summary}

## Generated by ISLS Forge

Crystal ID: {crystal_id}
Template: {template_name}
Generated: {timestamp}
```

Part IV
Workspace Intelligence

7 Understanding Existing Code

Definition 7.1 (WorkspaceAnalyzer)**.** The WorkspaceAnalyzer scans an existing project directory and builds a model of what exists:

```
pub struct WorkspaceAnalyzer;

impl WorkspaceAnalyzer {
    pub fn analyze(dir: &Path) -> Result<WorkspaceModel>;
}
```

```
pub struct WorkspaceModel {
    pub project_name: String,
    pub crate_type: CrateType,          // bin, lib, workspace
    pub modules: Vec<ModuleInfo>,
    pub types: Vec<TypeInfo>,
    pub traits: Vec<TraitInfo>,
    pub functions: Vec<FunctionInfo>,
    pub dependencies: Vec<DepInfo>,
    pub test_count: usize,
    pub loc: usize,                     // lines of code
    pub file_tree: Vec<String>,
}

pub struct ModuleInfo {
    pub name: String,
    pub path: String,
    pub public_items: Vec<String>,      // pub fn, pub struct, pub trait
    pub imports: Vec<String>,
    pub loc: usize,
}

pub struct TypeInfo {
    pub name: String,
    pub kind: TypeKind,                 // struct, enum, type alias
    pub fields: Vec<String>,
    pub derives: Vec<String>,
    pub module: String,
}

pub struct TraitInfo {
    pub name: String,
    pub methods: Vec<String>,
    pub implementors: Vec<String>,
    pub module: String,
}

pub struct FunctionInfo {
    pub name: String,
    pub signature: String,
    pub is_public: bool,
    pub has_test: bool,
    pub module: String,
}
```

Definition 7.2 (Workspace Context for Oracle)**.** When the Oracle synthesizes code for an existing project, it receives a WorkspaceContext:

```
pub struct WorkspaceContext {
    pub summary: String,                // "Axum REST API, 5 modules, 23
        types"
    pub relevant_types: Vec<TypeInfo>,  // types the new code needs
    pub relevant_traits: Vec<TraitInfo>,
    pub relevant_imports: Vec<String>,  // import paths for above
    pub existing_patterns: Vec<String>, // code patterns already used
    pub file_being_modified: Option<String>, // current file content
}
```

The Oracle prompt includes the workspace context so it generates code that uses existing types, follows existing patterns, and imports from existing modules — not code that reinvents what already exists.

8 Incremental Operations

Definition 8.1 (Incremental Forge Operations)**.** The Foundry supports incremental operations on existing projects:

```
pub enum FoundryOperation {
    /// Generate a complete new project from DecisionSpec
    NewProject {
        spec: DecisionSpec,
        output_dir: PathBuf,
    },

    /// Add a new module/component to an existing project
    AddComponent {
        project_dir: PathBuf,
        spec: String,           // "add a user authentication module"
        target_module: Option<String>, // where to add it
    },

    /// Implement a trait for an existing type
    Implement {
        project_dir: PathBuf,
        trait_name: String,
        type_name: String,
        hints: Option<String>,
    },

    /// Add an endpoint to an existing API
    AddEndpoint {
        project_dir: PathBuf,
        method: String,         // GET, POST, PUT, DELETE
        path: String,           // "/users/{id}"
        description: String,
    },

    /// Fix a specific file (rewrite with corrections)
    Fix {
        project_dir: PathBuf,
        file_path: String,
        issue: String,          // "doesn't compile", "test fails", etc.
    },

    /// Generate tests for existing code
    GenerateTests {
        project_dir: PathBuf,
        target: Option<String>, // specific module or all
    },

    /// Refactor a module
    Refactor {
        project_dir: PathBuf,
```

```
        target: String,
        instruction: String,
    },

    /// Generate documentation
    Document {
        project_dir: PathBuf,
        target: Option<String>,
    },
}
```

Part V
Quality Gates

9 The Extended Validation Pipeline

Definition 9.1 (Foundry Validation Pipeline)**.** For code artifacts, the validation pipeline extends beyond the 8 formal gates:

```
pub struct FoundryValidation {
    // Standard 8-gate (from C5)
    pub formal: FormalValidationResult,     // 8/8 checks

    // Foundry-specific gates
    pub compiles: bool,                     // cargo check passes
    pub tests_pass: bool,                   // cargo test passes
    pub test_count: usize,                  // number of tests
    pub warnings: usize,                    // cargo clippy warnings
    pub formatted: bool,                    // rustfmt check passes
    pub docs_build: bool,                   // cargo doc succeeds

    // Metrics
    pub loc: usize,                         // lines of code
    pub test_coverage_estimate: f64,        // tests/public_functions
    pub compilation_attempts: usize,        // how many tries needed
    pub oracle_tokens_used: usize,
}
```

Requirement 9.1 (Minimum Quality for Crystallization)**.** A code artifact is crystallized only if:

1. All 8 formal gates pass.
2. `cargo check` succeeds (zero compile errors).
3. `cargo test` succeeds (all tests pass).
4. At least one test exists per public function.
5. Zero Clippy errors (warnings are tolerated).

Artifacts that compile but have failing tests are *not* crystallized — they remain as "draft" artifacts in the project directory.

Part VI
Fabrication Modes

10 Interactive Mode

Interactive fabrication session

```
$ isls foundry new --domain rust
ISLS Foundry v1.0.0
> What do you want to build?
REST API for a bookmark manager with tags and search

> Template matched: T01 (REST API Service)
> Decomposing into 9 atoms across 3 molecules...
> Forging atom 1/9: router... [Oracle] ... OK (1.2s)
> Forging atom 2/9: middleware... [Memory] ... OK (0.01s)
> Forging atom 3/9: error_handler... [Oracle] ... OK (0.8s)
> Forging atom 4/9: models... [Oracle] ... OK (1.5s)
> Forging atom 5/9: service... [Oracle] ... OK (2.1s)
> Forging atom 6/9: dto... [Derive] ... OK (0.1s)
> Forging atom 7/9: database... [Oracle] ... OK (1.8s)
> Forging atom 8/9: config... [Static] ... OK (0.01s)
> Forging atom 9/9: main... [Static] ... OK (0.01s)

> Writing project to ./bookmark-api/
> Running cargo check... FAIL (attempt 1/5)
  error[E0599]: no method named `fetch_all` on `SqlitePool`
> Feeding error to Oracle... corrected.
> Running cargo check... OK
> Running cargo test... 12 tests, 12 passed
> Running cargo clippy... 0 errors, 2 warnings
> Running cargo fmt... formatted

> Fabrication complete.
  Project: ./bookmark-api/
  Crystal: a7b3c9...
  Files: 12  |  LOC: 847  |  Tests: 12
  Attempts: 2  |  Tokens: 8,420
  Template: T01 (REST API)
  Autonomy: 33% (3/9 from memory/static)
```

11 Batch Mode

```
# Generate from a DecisionSpec file
isls foundry new --spec bookmark-api.json --output ./bookmark-api/

# Add to existing project
isls foundry add --project ./bookmark-api/ \
  --spec "add a /search endpoint with full-text search on title and tags
     "

# Fix existing code
```

```
isls foundry fix --project ./bookmark-api/ \
  --file src/database.rs --issue "add pagination to list_bookmarks"

# Generate tests for untested code
isls foundry test --project ./bookmark-api/

# Refactor
isls foundry refactor --project ./bookmark-api/ \
  --target src/service.rs --instruction "extract validation into a
     separate module"
```

12 Headless Mode (for automation)

```
# JSON in, JSON out (for CI/CD or other tools)
echo '{"operation":"new","spec":{...},"output":"./out/"}' | \
  isls foundry --json

# Returns:
{
  "success": true,
  "project_dir": "./out/",
  "crystal_id": "a7b3c9...",
  "files": 12,
  "loc": 847,
  "tests": 12,
  "attempts": 2,
  "tokens": 8420
}
```

Part VII
Rust API

```
// isls-foundry/src/lib.rs

pub struct Foundry {
    forge: ForgeEngine,
    oracle: OracleEngine,
    catalog: TemplateCatalog,
    toolchain: ToolchainExecutor,
    analyzer: WorkspaceAnalyzer,
    config: FoundryConfig,
}

pub struct FoundryConfig {
    pub max_attempts: usize,          // default: 5
    pub require_tests: bool,          // default: true
    pub require_clippy_clean: bool,   // default: false (warnings ok)
    pub require_fmt: bool,            // default: true
    pub auto_fmt: bool,               // default: true
    pub generate_readme: bool,        // default: true
```

```
    pub generate_gitignore: bool,     // default: true
    pub dry_run: bool,                // skip compilation
}

pub struct FabricationResult {
    pub success: bool,
    pub project_dir: PathBuf,
    pub crystal: Option<SemanticCrystal>,
    pub files: Vec<GeneratedFile>,
    pub validation: FoundryValidation,
    pub attempts: usize,
    pub tokens_used: usize,
    pub duration_ms: u64,
    pub template_used: Option<String>,
    pub autonomy_ratio: f64,
}

pub struct GeneratedFile {
    pub path: String,
    pub content: String,
    pub source: SynthesisSource,      // Oracle, Memory, Static, Derive
    pub loc: usize,
    pub test_count: usize,
}

impl Foundry {
    pub fn new(config: FoundryConfig, forge: ForgeEngine,
               oracle: OracleEngine, catalog: TemplateCatalog) -> Self;

    /// Full fabrication: spec -> working project on disk
    pub fn fabricate(
        &mut self, op: FoundryOperation
    ) -> Result<FabricationResult>;

    /// Analyze an existing workspace
    pub fn analyze(&self, dir: &Path) -> Result<WorkspaceModel>;

    /// Check if toolchain is available
    pub fn toolchain_available(&self) -> bool;
}
```

Part VIII
Gateway Integration

New Gateway Endpoints

```
POST /foundry/new          Body: {spec, domain, output_dir}
                           -> FabricationResult

POST /foundry/add          Body: {project_dir, spec}
                           -> FabricationResult (incremental)

POST /foundry/fix          Body: {project_dir, file, issue}
```

```
                          -> FabricationResult (fix)

POST /foundry/test        Body: {project_dir, target?}
                          -> FabricationResult (test generation)

GET  /foundry/analyze     Params: ?dir=path
                          -> WorkspaceModel

GET  /foundry/status      -> current fabrication progress
```

Part IX
Acceptance Tests

ID	Name	Procedure
AT-FD1	New project compiles	Fabricate REST API from spec; verify `cargo check` passes.
AT-FD2	Tests pass	Fabricate project; verify `cargo test` passes with $\geq$ 1 test per public fn.
AT-FD3	Compile-fix loop	Mock Oracle to return code with missing import on first call, fixed on second; verify 2 attempts, success.
AT-FD4	Budget exhaustion	Set max_attempts=1; mock Oracle to return broken code; verify failure with diagnostics.
AT-FD5	Workspace analysis	Create a Rust project with 3 modules; run analyzer; verify correct module/type/function counts.
AT-FD6	Incremental add	Create project; add endpoint via Foundry; verify new route in router and `cargo check` still passes.
AT-FD7	Fix operation	Create project with compile error; run fix; verify error resolved.
AT-FD8	Test generation	Create project with untested functions; run test generation; verify tests created and passing.
AT-FD9	Scaffold completeness	Fabricate project; verify Cargo.toml, README.md, .gitignore, src/, tests/ all present.
AT-FD10	Dry run mode	Fabricate with dry_run=true; verify files written but no cargo commands executed.

AT-FD11	Workspace context	Add endpoint to existing project; verify Oracle prompt includes existing types and imports.
AT-FD12	Crystal contains validation	Fabricate project; inspect crystal; verify it records compilation_attempts and test_count.
AT-FD13	Formatted output	Fabricate project; verify all .rs files pass `rustfmt --check`.
AT-FD14	README generated	Fabricate project; verify README.md contains project description, architecture, and getting started.

Part X
Handoff

13 Test Count

Phase	New Tests	Cumulative
Phase 1–7 + Genesis	265	265
Phase 8 (C27)	14	$\geq$ 279

14 Completion Criteria

1. Fabricating a new project from DecisionSpec produces a directory with Cargo.toml, src/, tests/, README.md.
2. Generated code compiles (`cargo check` passes).
3. Generated tests pass (`cargo test` passes).
4. Compile-fix loop works: Oracle corrects errors on retry.
5. Workspace analyzer correctly models existing projects.
6. Incremental operations (add, fix, test, refactor) work on existing projects.
7. Crystal records compilation status and test count.
8. Dry-run mode works without `cargo` installed.
9. Total tests $\geq$ 279, zero warnings.
10. **27 Crates. The Forge that compiles its own output.**

Generated for ISLS Extension Phase 8 v1.0.0.
The Oracle generates. The compiler verifies. The loop closes.
Code that doesn't compile is not a crystal. Code that doesn't test is not complete. Code that doesn't fit is not useful.

The Foundry makes all three guarantees.